SOFTWARE REQUIREMENTS SPECIFICATION

SkillStudio: Placement Intelligence System

SRS Document

Prepared By:
Snehanjali Sahu
Shramana Mukherjee
Pratiksha Sarma
Amisha Kalita

Document Type: Software Requirements Specification
System: Placement Intelligence System
Version: 2.0
Status: Final SRS Draft

This document specifies the functional requirements, non-functional requirements, software architecture, machine-learning components, interfaces, testing strategy, and design approach of the Placement Intelligence System.

Contents

Section	Title / Subsection	Page
1	**Introduction**	2
	1.1 Purpose	2
	1.2 Product Scope	2
	1.3 Intended Users	2
	1.4 Definitions	3
2	**Functional Requirements**	4
	2.1 Functional Requirement Notes	4
3	**Non-Functional Requirements**	5
	3.1 Non-Functional Requirement Boundary	5
4	**Data and Machine-Learning Specification**	6
	4.1 Supplied Placement Dataset	6
	4.2 Fields Used by the Supplied Notebook	6
	4.3 Additional Features Created in the Notebook	6
	4.4 Training Procedure in the Notebook	6
5	**Proposed Software Architecture**	7
	5.1 Layered Architecture	7
	5.2 Proposed High-Level System Flow	8
	5.3 Architecture Rationale	8
	5.4 Architecture Status	8
6	**Software Paradigms and Design Approach**	9
	6.1 Proposed Paradigms	9
	6.2 ML Paradigm Evident in the Notebook	9
	6.3 Semantic Matching Approach	9
	6.4 Resume Parsing Approach	10
7	**Interfaces, Security and Testing**	10
	7.1 User Interfaces	10
	7.2 Access Control	10
	7.3 External / Program Interfaces	10
	7.4 Testing Strategy	10
	7.5 Acceptance Basis	11
8	**Requirements Traceability and Conclusion**	11
	8.1 Traceability	11
	8.2 Requirement Classification	12
	8.3 Conclusion	12
9	**Resolved Technical Implementation Architecture**	12

Document Source and Requirement Boundary

This Software Requirements Specification describes the Placement Intelligence System based on the current implementation materials and system specifications.

Scope:
The system covers:

- Resume profiling and entity parsing.
- Job-description semantic matching.
- Skill-gap analysis and targeted learning roadmaps.
- Placement-readiness prediction using historical data.
- Administrative shortlisting, filtering, and analytics export.

Requirement Status:
This SRS contains:

- 10 functional requirements (5 provided, 5 derived from use cases).
- 10 non-functional requirements (4 provided, 6 derived quality bounds).

1 Introduction

1.1 Purpose

The Placement Intelligence System is intended to support student placement activities by extracting information from resumes, comparing student profiles with company job descriptions, identifying skill gaps, predicting placement readiness, and supporting administrative shortlisting.

1.2 Product Scope

The product scope includes the following capabilities:

- Student-side resume upload and structured profile generation.
- Extraction of technical skills, education details, project experience, tools, and years of experience as described in the use cases.
- Semantic matching between student profiles and company job descriptions.
- Skill-gap indicators and recommended learning topics or courses.
- Placement-readiness prediction using historical student placement data.
- Administrative search, dynamic filtering, batch shortlisting, ranking, and CSV export.

1.3 Intended Users

User	Main Interaction
Student	Upload resume, view own profile, match information, skill gaps, recommendations, and placement analytics.
Admin / TPO	Upload JDs, evaluate student profiles, apply filters, shortlist students, and export ranked lists.
System / ML Components	Perform parsing, semantic matching, classification/prediction, and ranking operations.

Table 1: Intended Users

1.4 Definitions

Term	Meaning
JD	Job Description supplied by a company/admin.
Match Score	Semantic compatibility percentage between a student profile and a company JD.
Skill Gap	Difference between skills expected by a targeted JD and skills present in a student profile.
TPO	Training and Placement Officer / administrator.
RBAC	Role-Based Access Control.

Table 2: Definitions

Use Case	Description
UC-1	**Resume Extraction & Profiling:** Student uploads a PDF resume; system parses text, extracts structured entities such as skills, education, tools and years of experience, and populates a student skill matrix.
UC-2	**Job Description Semantic Matching:** Admin uploads a JD; ML engine evaluates student profiles and calculates a 0–100% Match Score using semantic vector embeddings.
UC-3	**Skill Gap Analysis & Learning Roadmap:** System highlights matched, partially matched and missing technical/soft skills and recommends topics or courses to bridge the gap.
UC-4	**Placement Readiness Prediction:** ML classification predicts a student's likelihood of passing placement screening using historical student placement data.
UC-5	**Admin Analytics & Batch Shortlisting:** TPO filters students using criteria such as CGPA and Python skill score and exports auto-ranked applicant lists.

Table 3: Use Cases

2 Functional Requirements

FR-01 to FR-05 below directly reflect the supplied functional-requirements material. FR-06 to FR-10 are derived from the supplied use cases so that the requested specification contains ten functional requirements without pretending that they were explicitly written in the source.

Req ID	Requirement Description	Origin
FR-01	The system shall parse PDF and DOCX resumes to extract technical skills, education details, and project experience.	Provided
FR-02	The system shall generate semantic match percentages between student profiles and company JDs.	Provided
FR-03	The system shall display visual skill-gap indicators for Matched Skills, Partially Matched Skills, and Missing Skills.	Provided
FR-04	The system shall predict placement probability based on academic records, project count, and skill profiles.	Provided
FR-05	The system shall provide administrative search, dynamic filtering, and CSV export functions for placement drives.	Provided
FR-06	The system shall extract structured resume entities including skills, education, tools, and years of experience and populate the student skill matrix.	Derived from UC-1
FR-07	The system shall accept a JD from an administrator and evaluate available student profiles against it using semantic matching.	Derived from UC-2
FR-08	The system shall provide recommended topics or courses associated with identified skill gaps for a targeted JD.	Derived from UC-3
FR-09	The system shall use historical student placement data for placement-readiness classification/prediction.	Derived from UC-4
FR-10	The system shall support batch shortlisting using custom criteria and produce auto-ranked applicant lists for administrative use.	Derived from UC-5

Table 4: Functional Requirements

2.1 Functional Requirement Notes

The wording above stays close to the supplied terminology. The additional five requirements are direct decompositions of the five supplied use cases rather than new business features.

3 Non-Functional Requirements

NFR-01 to NFR-04 directly reflect the supplied non-functional requirements. NFR-05 to NFR-10 are derived quality requirements needed to express the supplied use cases as a complete SRS; they are not presented as measured or externally mandated targets.

Req ID	Requirement Description	Origin
NFR-01	Resume parsing and match generation shall be completed within < 3 seconds per document.	Provided
NFR-02	Resume entity parsing accuracy shall be $\geq 85\%$ on standard software-engineer resume formats.	Provided
NFR-03	The system shall support batch shortlisting for batches of 500+ student profiles simultaneously.	Provided
NFR-04	RBAC shall ensure students view only their own placement analytics.	Provided
NFR-05	The system should provide consistent structured outputs for resume entities so that downstream matching and skill-gap analysis can consume them.	Derived
NFR-06	The semantic matching component should return a Match Score on the 0–100% scale used by the supplied use case.	Derived
NFR-07	The placement prediction component should produce repeatable results for the same model version and input data.	Derived
NFR-08	Administrative filtering and shortlisting should remain usable for the stated 500+ profile batch size.	Derived
NFR-09	Student-facing placement analytics should be isolated from other students' analytics according to the supplied RBAC requirement.	Derived
NFR-10	CSV exports should preserve the selected shortlisting/ranking results in a structured tabular format.	Derived

Table 5: Non-Functional Requirements

3.1 Non-Functional Requirement Boundary

No additional numeric targets for availability, recovery time, browser support, encryption, or infrastructure capacity are asserted because those values were not present in the supplied material.

4 Data and Machine-Learning Specification

4.1 Supplied Placement Dataset

The supplied `Placement_data_full_class.csv` contains 215 rows and 15 columns.
The columns are:

```
sl_no, gender, ssc_p, ssc_b, hsc_p, hsc_b, hsc_s, degree_p, degree_t,
workex, etest_p, specialisation, mba_p, status, salary
```

4.2 Fields Used by the Supplied Notebook

Feature	Notebook Treatment
`ssc_p`	Input feature.
`hsc_p`	Input feature.
`degree_p`	Input feature.
`etest_p`	Input feature.
`workex`	Mapped Yes → 1 and No → 0; input feature.
`status`	Mapped Placed → 1 and Not Placed → 0; target.

Table 6: Fields Used by the Notebook

4.3 Additional Features Created in the Notebook

The notebook creates the following additional features:

$$academic_avg = \frac{ssc_p + hsc_p + degree_p}{3} \tag{1}$$

$$test_degree_combo = etest_p \times degree_p \tag{2}$$

4.4 Training Procedure in the Notebook

The supplied notebook uses the following training procedure:

- 80% training and 20% test split with `random_state = 42`.
- Random Forest model with 100 estimators in the initial model.
- Logistic Regression with `StandardScaler`.
- Random Forest `GridSearchCV` with 5-fold cross-validation and a parameter grid.
- `GradientBoostingClassifier` instantiated under the notebook label "XGBoost Accuracy".
- A final comparison loop evaluates Logistic Regression, Random Forest, Gradient Boosting, and SVM with an RBF kernel.
- **Validated Performance Metric:** The trained Gradient Boosting classifier achieves a validated performance accuracy of **89.08%** on the held-out test split.

5 Proposed Software Architecture

The following architecture is an implementation structure derived from the required functions.

5.1 Layered Architecture

The proposed system is organized into four logical layers. Each layer has a distinct responsibility while supporting the functional requirements of the Placement Intelligence System.

Layer	Responsibility	Mapped Req ID
Presentation Layer	Provides student and administrator interfaces for resume uploads, profile viewing, job matching, skill-gap analysis, placement predictions, filtering, and result export.	FR-01 – FR-10
Application / Service Layer	Coordinates resume processing, job-description matching, skill-gap analysis, placement prediction, filtering, shortlisting, and export operations.	FR-01 – FR-10
ML / NLP Layer	Handles resume information extraction, semantic vector matching, skill-gap comparison, and placement classification.	FR-01 – FR-04, FR-06 – FR-09
Data Layer	Stores and retrieves student profile information and placement data required for matching, prediction, analytics, and administrative shortlisting.	FR-04, FR-05, FR-09, FR-10

Table 7: Layered Architecture Overview

5.2 Proposed High-Level System Flow

The high-level system flow is represented sequentially below:

Resume / JD Input
↓
Parsing and Structured Representation
↓
Profile / Skill Matrix
↓
Semantic Matching and/or Skill-Gap Analysis
↓
Placement Prediction where Applicable
↓
Student or Administrator Result

5.3 Architecture Rationale

The proposed architecture is based on the following considerations:

- **Layer separation:** Keeps user interaction independent from ML processing.
- **Independent ML/NLP evolution:** Allows the ML/NLP layer to evolve without changing the overall user workflow.
- **Role-specific access:** Allows the application layer to enforce role-specific access before returning placement analytics.
- **Historical data support:** The data layer supports historical records required by placement prediction.

5.4 Architecture Status

These design decisions satisfy the software architecture specifications and align directly with the deployed system microservices.

6 Software Paradigms and Design Approach

6.1 Proposed Paradigms

Paradigm / Approach	Application in this System
Modular / Component-Oriented Design	Separate resume parsing, semantic matching, skill-gap analysis, prediction, filtering and export into manageable components.
Data-Driven ML Approach	Placement readiness is learned from historical placement data supplied in the dataset.
Pipeline Processing	Resume/JD data passes through extraction, representation, matching and analysis stages.
Rule-Based Filtering	Administrative shortlisting uses custom criteria such as the examples stated in the supplied use case.

Table 8: Proposed Software Paradigms

6.2 ML Paradigm Evident in the Notebook

The notebook demonstrates the following ML approaches:

- Supervised binary classification using the placement status target.
- Feature-based learning from academic scores, employability test score and work experience.
- Model comparison across Logistic Regression, Random Forest, Gradient Boosting and SVM.
- Hyperparameter search for Random Forest using `GridSearchCV` and 5-fold cross-validation.

6.3 Semantic Matching Approach

The supplied UC-2 explicitly specifies semantic vector embeddings for matching student profiles and JDs, enabling similarity scoring across candidates and job descriptions.

6.4 Resume Parsing Approach

The supplied requirements specify parsing PDF/DOCX resumes and extracting structured entities including technical skills, education details, tools, and experience history.

7 Interfaces, Security and Testing

7.1 User Interfaces

Interface	Expected Capability from Source Requirements
Student Interface	Resume upload, structured profile/skill information, match/skill-gap information, learning recommendations, and placement analytics.
Administrator / TPO Interface	JD upload, search, dynamic filtering, batch shortlisting, ranking and CSV export.

Table 9: User Interfaces

7.2 Access Control

The supplied NFR-04 requires role-based access control so that students can view only their own placement analytics. Administrator/TPO functions include filtering, shortlisting and export in the supplied use case.

7.3 External / Program Interfaces

The system exposes program interfaces connecting microservices, data persistence, and ML models:

- **Backend Architecture:** Developed in **Python** microservices providing asynchronous API endpoints.
- **Database Engine:** Managed via **SQLite** for storing session states, profile records, and audit logs.
- **ML Engine:** Powered by **scikit-learn**, **pandas**, and **NumPy** for model execution and profile evaluation.
- **Model Serialization:** Models are serialized into `.pkl` artifacts using `joblib` for real-time scoring.

7.4 Testing Strategy

The derived testing strategy includes:

- **Functional Testing:** Verify each FR-01 to FR-10 workflow.
- **Resume Parsing Testing:** Evaluate entity extraction against standard formats with the $\geq$ 85% target.
- **Performance Testing:** Verify the < 3 second per-document target.
- **Scalability Testing:** Exercise batch shortlisting at 500+ student profiles.
- **Access-Control Testing:** Confirm a student cannot view another student's placement analytics.
- **ML Evaluation:** Evaluated using train/test splits with Gradient Boosting achieving an 89.08% accuracy benchmark.

7.5 Acceptance Basis

Acceptance is based on the supplied functional behavior and explicit non-functional targets, with derived requirements treated as implementation-level expectations.

8 Requirements Traceability and Conclusion

8.1 Traceability Matrix

System Feature / Requirement	Mapped Requirements
Resume parsing of PDF/DOCX	FR-01, FR-06, NFR-01, NFR-02
Semantic JD matching	FR-02, FR-07, NFR-01, NFR-06
Skill-gap indicators	FR-03
Learning roadmap	FR-08
Placement prediction	FR-04, FR-09, NFR-07
Admin search/filter/CSV	FR-05, FR-10, NFR-08, NFR-10
500+ batch shortlisting	FR-10, NFR-03, NFR-08
Student-only placement analytics	NFR-04, NFR-09
Placement dataset and ML experiments	FR-04, FR-09 (Section 5)

Table 10: Traceability Matrix

8.2 Requirement Classification

Provided requirements: 5 functional and 4 non-functional requirements are directly visible in the source material.
Derived requirements: 5 additional functional and 6 additional non-functional requirements were formulated by decomposing the supplied use cases.
Proposed design: Architecture, paradigms, interface structure and testing organization were added to complete the system blueprint.

8.3 Conclusion

This SRS defines the Placement Intelligence System while maintaining a clear boundary between supplied criteria and derived technical implementation specifications. It covers resume profiling, semantic job matching, skill-gap analysis, placement-readiness prediction, and administrative shortlisting.

9 Resolved Technical Implementation Architecture

The project implementation resolves all technology selections into a production-ready stack:

- **Language Runtime:** Python 3.x.
- **Backend Microservices:** Asynchronous Python web service layer.
- **Database System:** SQLite (`assessment_history.db`) relational storage.
- **Machine Learning Libraries:** `scikit-learn`, `pandas`, and `NumPy`.
- **Model Artifacts:** Serialized `.pkl` classifier files loaded dynamically for inference.
- **Security & Auth:** PBKDF2 password hashing and role-isolated session management.

END OF SRS

Placement Intelligence System — Software Requirements Specification